

UNIVERSITY OF SCIENCE - VNUHCM
FACULTY OF INFORMATION TECHNOLOGY



OBJECT-ORIENTED PROGRAMMING REPORT

SUBJECT: MANAGEMENT SYSTEM APPLICATION

Instructor:

M.S. Võ Nhật Tân

M.S. Trương Tấn Khoa

Students:

Dương Anh Hào - 25127045

Huỳnh Phúc Nguyên - 25127106

Lâm Hồng Hải Hoàng Lê - 25127207

Nguyễn Việt Bách - 25127275

Đỗ Minh Khang - 25127363

Thành phố Hồ Chí Minh, tháng 8 năm 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our lecturers Trương Tấn Khoa and Võ Nhật Tân for the invaluable guidance, encouragement, and support provided throughout the development of this Object-Oriented Warehouse Management System. Through this project, we had the opportunity to apply the theoretical concepts learned in the Object-Oriented Programming course to a practical software engineering problem. The knowledge and feedback provided by our lecturer not only helped us understand object-oriented principles more deeply but also encouraged us to adopt good software development practices, including modular design, code reusability, collaborative development, and systematic problem-solving. We are also grateful for the constructive comments and suggestions received during the project's development, which motivated us to continuously refine both the system implementation and this report. In addition, we would like to express our appreciation to our teammates for their dedication, responsibility, and effective collaboration throughout the project. The successful completion of the Warehouse Management System was the result of continuous communication, mutual support, and the collective effort of every team member in overcoming technical challenges and integrating different system components. This project has been a valuable learning experience, allowing us to strengthen not only our technical knowledge in object-oriented programming, software architecture, database integration, and version control, but also essential soft skills such as teamwork, communication, project planning, and time management. Although we have devoted considerable effort to ensuring the quality and completeness of both the developed system and this report, we recognize that there are still limitations and opportunities for future improvement. We sincerely welcome any comments, suggestions, and constructive feedback that may help us further enhance our work and continue developing our knowledge and skills.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS.....	2
TABLE OF CONTENTS.....	3
LIST OF FIGURES.....	6
LIST OF TABLES.....	7
GLOSSARY.....	8
1. GROUP INTRODUCTION.....	9
1.1. General information.....	9
1.2. Member roles and task assignments.....	10
2. PROBLEM STATEMENT.....	11
2.1. Selected management topic.....	11
2.2. Target Users.....	11
2.3. Main features.....	11
2.4. Input and output data.....	12
3. SYSTEM DESIGN.....	13
3.1. UML Class Diagram.....	13
3.2. Main classes and relationships.....	13
3.2.1. Package Data Group.....	14
3.2.1.1. Package - Central Domain Class.....	14
3.2.1.2. PackageMetadata.....	14
3.2.1.3. Address.....	15
3.2.1.4. StorageLocation.....	15
3.2.1.5. LogisticsInfo.....	15
3.2.1.6. Dimensions.....	15
3.2.1.8. Typedef Date.....	16
3.2.2. Package Stage Group.....	16
3.2.2.1. Interface IPackageState.....	16
3.2.2.2. Enum PackageStateId.....	17

3.2.2.3. Concrete State Classes.....	17
3.2.3. Repository and Filter Group.....	18
3.2.3.1. Interface IPackageRepository.....	18
3.2.3.2. PackageQueryCriteria.....	19
3.2.3.3. JsonPackageRepository.....	19
3.2.3.4. SqlitePackageRepository.....	20
3.2.3.5. DatabaseConnection.....	20
3.2.4. WarehouseManager.....	21
3.2.5. Presentation Layer (GUI).....	22
3.2.5.1. MainWindow.....	23
3.2.5.2. FilterPanel.....	26
3.2.5.3. PackageTableModel.....	26
3.2.5.4. AddPackageDialog.....	27
3.2.5.5. EditPackageDialog.....	27
3.2.5.6. WarehouseGateway.....	28
3.2.5.7. Data flow summary.....	29
3.2.6. Relationship Summary.....	29
3.3. Application flow or main use-case flow.....	30
3.3.1. New Package Addition.....	30
3.3.2. Lifecycle State Transitions.....	31
3.3.3. Querying and Filtering.....	32
3.3.4. Packages Update and Removal.....	33
3.3.5. GUI synchronization.....	33
4. PROJECT ARCHITECTURE.....	34
4.1. Project Structure.....	34
4.2. Architecture.....	36
4.3. Design Patterns.....	37
5. IMPLEMENTATION AND TESTING.....	39

5.1. Main modules.....	39
5.2. Data storage method.....	39
5.3. Important features.....	40
5.4. Test cases and results.....	40
5.5. Bugs found and fixes.....	43
6. AI USAGE AND INDIVIDUAL ACCOUNTABILITY.....	45
6.1. AI tools.....	45
6.2. Individual AI usage log of each member.....	45
6.3. Individual contribution.....	46
7. CHALLENGES AND FUTURE ENHANCEMENTS.....	46
7.1. Encountered Problems.....	46
7.2. Limitations of the current version.....	47
7.3. Possible future improvements.....	47
REFERENCES.....	48

LIST OF FIGURES

Image 1. UML Warehouse management system..... 13

Image 2. State Lifecycle..... 31

Image 3. Warehouse Management Architecture..... 36

LIST OF TABLES

Table 1: Members’ information..... 9

Table 2: Contribution Table..... 10

Table 3. MainWindow Modules table.....25

Table 4. Folder Structure.....34

Table 5. Project’s Design Patterns..... 37

Table 6. Main modules.....39

Table 8. Bugs and fixes.....43

Table 9. AI tools usage.....45

GLOSSARY

- **UUID (Universally Unique Identifier):** a 128-bit identifier, generated here as a version-4 (random) UUID by `Package::generateUuid()`, used as the stable primary key for a `Package` across both the JSON file and the SQLite id column.
- **RAII (Resource Acquisition Is Initialization):** the C++ idiom of tying a resource's lifetime to an object's constructor/destructor, applied by `DatabaseConnection` to guarantee its `QSqlDatabase` connection is opened on construction and always closed and deregistered on destruction.
- **CRUD (Create, Read, Update, Delete):** the four basic persistence operations defined on `IPackageRepository` (`add`, `getAll/getById`, `update`, `remove`) and exposed one level up through `WarehouseManager`'s package-management methods.

1. GROUP INTRODUCTION

1.1. General information

Table 1: Members' information

Group ID: 01		
Student ID	Full Name	Email
25127045	Dương Anh Hào	dahao2532@clc.fitus.edu.vn
25127106	Huỳnh Phúc Nguyên	hpnguyen2527@clc.fitus.edu.vn
25127207	Lâm Hồng Hải Hoàng Lê	lhhhle2508@clc.fitus.edu.vn
25127275	Nguyễn Viết Bách	nvbach2517@clc.fitus.edu.vn
25127363	Đỗ Minh Khang	dmkhang2525@clc.fitus.edu.vn

1.2. Member roles and task assignments

Table 2: Contribution Table

Full Name	Assigned Tasks	Role	Contribution (%)
Dương Anh Hào	- CMAKE implementation and program merge - GUI Design - Sub-system implementation	Member	100%
Huỳnh Phúc Nguyên	- Task Tracking - Report writing - Slide design - Core system implementation	Vice project manager	100%
Lâm Hồng Hải Hoàng Lê	- Core GUI design - Sub-system implementation - Main tester	Member	100%
Nguyễn Viết Bách	- GUI design - Video recording - Sub-system implementation	Member	100%
Đỗ Minh Khang	- Github tracking - UML design - Core system implementation - Polish GUI design - Video recording	Project manager	100%

2. PROBLEM STATEMENT

2.1. Selected management topic

A Warehouse Management System (WMS) has been selected as the core topic for this project to simulate and solve common inventory tracking challenges through an object-oriented approach. In traditional manual systems, reliance on paper records or simple spreadsheets often leads to frequent data discrepancies, delayed updates during stock intake, and general inefficiencies. For an academic programming project, this domain provides a highly practical environment to demonstrate four Object-Oriented Programming (OOP) concepts: **Encapsulation, Inheritance, Polymorphism, and Abstraction**. Furthermore, to ensure the software architecture is scalable and follows clean coding standards, the system will integrate at least two design patterns, specifically the **State Pattern** and the **Observer Pattern**, which will decouple object creation and behavioral interactions within the system.

2.2. Target Users

The system categorizes target users into three main operational roles to establish basic access control and reflect a standard warehouse environment.

- + **Inventory Managers** oversee the general state of the warehouse by monitoring stock levels, approving inbound or outbound requests, defining safety thresholds for reordering, and viewing system reports.
- + **Warehouse Staff** handle day-to-day operations on the floor, interacting directly with the program to perform physical stocktakes, update item storage locations, and log new stock movements.

2.3. Main features

The proposed Warehouse Management System provides several key features to support efficient warehouse operations and package lifecycle management.

- + **Package Registration:** Allows warehouse staff to record new packages together with their metadata, storage location, logistics information, and source and destination addresses.
- + **Package Tracking:** Enables users to monitor the current status and storage location of packages throughout their lifecycle, from arrival at the warehouse until dispatch.
- + **Inventory Monitoring:** Enables warehouse staff to monitor inventory levels and warehouse utilisation to support daily operations and capacity planning.
- + **Package Search:** Supports efficient retrieval of package information using multiple search, allowing users to quickly locate packages and identify specific groups of interest.
- + **Warehouse Exception Handling:** Assists warehouse staff in identifying and managing exceptional situations such as overdue or missing packages to improve operational reliability.

2.4. Input and output data

Input data is acquired from two sources. The primary source is user interaction through the graphical user interface, where package information—including metadata, source and destination addresses, logistics details, and storage location—is entered or modified using dedicated package management dialogs. The secondary source is the persistence layer, from which previously stored package records are automatically loaded during application initialization through either the JSON- or SQLite-based repository implementation.

Output data is delivered through the graphical user interface, where package information, warehouse occupancy, package state distribution, and analytical reports are presented to support warehouse monitoring and management. In addition, all modifications are persisted to the selected storage mechanism, ensuring that the current warehouse state is maintained consistently and remains available across subsequent application sessions.

3.1. UML Class Diagram

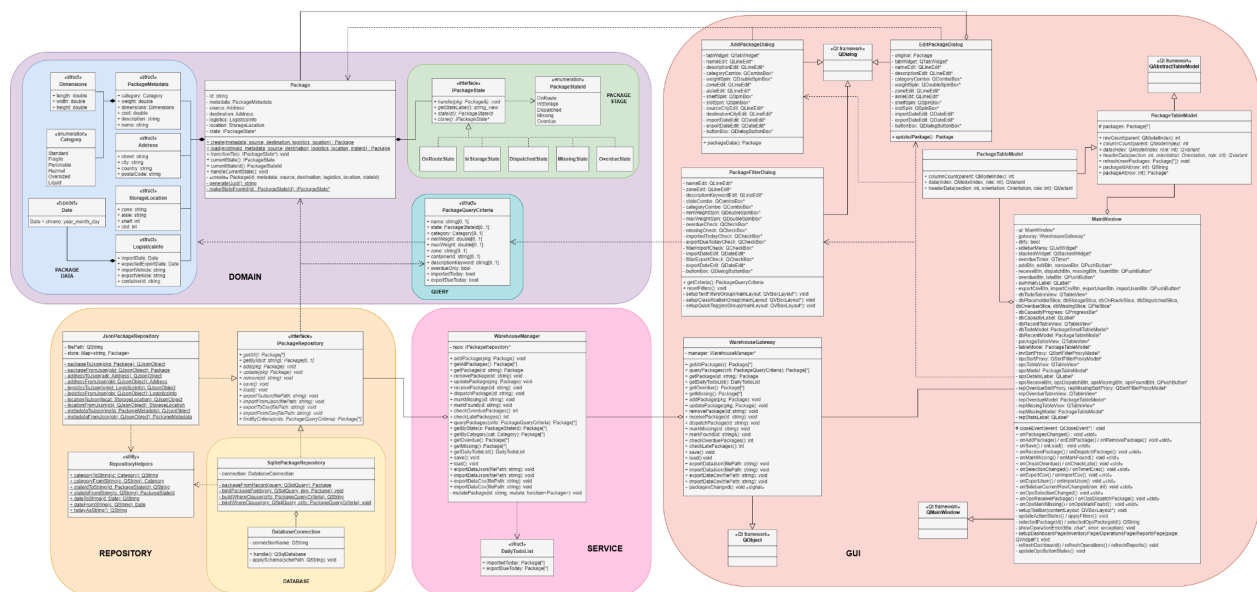


Image 1. UML Warehouse management system

3.2. Main classes and relationships

The system is organized into four cohesive component groups:

- + **Package Data** constitutes the core domain model, capturing all information that fully describes a physical package in the warehouse.
- + **Package Stage** governs the package's lifecycle by implementing the State Pattern, allowing each phase of a package's journey to encapsulate its own behaviour and transition logic.
- + **Repository and Filter** provides the persistence and query layer, isolated from business logic through abstract interfaces so that the storage backend can be changed without affecting any other part of the system.
- + **GUI (Presentation)** renders the domain and service layer through a Qt Widgets front end, translating user actions into WarehouseManager calls and reflecting the resulting state back on screen.

3.2.1. Package Data Group

This group contains the classes responsible for representing a package in its entirety. Rather than consolidating all attributes into a single class, the design composes **Package** from several smaller and purpose-specific value objects. Each object **encapsulates** a distinct concern — **description, address, logistics, location, and size** — so that concerns can evolve independently without forcing unrelated changes elsewhere.

3.2.1.1. Package - Central Domain Class

The **Package** class is the central domain entity and aggregate root, encapsulating all related value objects while maintaining ownership of the current lifecycle state. Its design separates entity identity (a generated UUID) from descriptive attributes (**PackageMetadata**), addressing (**Address**), physical placement (**StorageLocation**), logistics (**LogisticsInfo**), and lifecycle state (**IPackageState**). This separation of concerns improves modularity and allows each component to evolve independently.

Object creation is restricted through a private constructor and two static factory methods. One creates a newly registered package by generating a new UUID and initializing it in the **OnRoute** state, while the other reconstructs an existing package from persistent storage by restoring its identifier and lifecycle state.

Since the lifecycle state is owned through `std::unique_ptr`, copy operations implement deep-copy semantics. The copy constructor and copy-assignment operator invoke the state's `clone()` method to create an independent state instance, preserving value semantics without sharing ownership.

3.2.1.2. PackageMetadata

PackageMetadata is a value object that encapsulates the descriptive information of a package, including its name, category, weight, dimensions, declared cost, and description. By separating these attributes from other aspects of the package, such as routing and logistics information, the design improves modularity and allows metadata to be validated or updated independently.

3.2.1.3. Address

Address represents a standard postal address, including the street, city, country, and postal code. The same **Address** class is used for both the sender and destination addresses of a package, avoiding the duplication that would arise from defining two separate address types. This design follows the DRY (Don't Repeat Yourself) principle, reduces code duplication, and ensures that any future changes to the address structure are applied consistently across both locations.

3.2.1.4. StorageLocation

StorageLocation encodes the precise physical position of a package within the warehouse using a four-level hierarchy: zone (a broad warehouse area, e.g. "A" or "Cold"), aisle (a row within the zone), shelf (a vertical level on the rack, typed as int), and slot (an individual position on the shelf, also int). This hierarchical structure maps directly onto real warehouse infrastructure and makes it straightforward to locate, reassign, or audit packages without ambiguity.

3.2.1.5. LogisticsInfo

LogisticsInfo captures the operational and timeline details associated with a shipment. The import date and expected export date use the `Date` alias — a type-safe, calendar-aware representation rather than an unvalidated string — while the import and export vehicle fields record the transport identifiers used inbound and outbound, and the container ID identifies the shipping container assigned to carry the package. Together, these fields support queries by vehicle or container and enable the detection of overdue packages by comparing the expected export date with the current date.

3.2.1.6. Dimensions

Dimension is a minimal value object composed into **PackageMetadata** to represent the three-dimensional physical size of a package, expressed in a consistent unit. Separating dimensional data into its own type keeps size-related validation concentrated in one place rather than scattered across the domain model.

3.2.1.7. Enum Category

Category is implemented as a strongly typed enumeration (enum class) to represent package classifications within the warehouse management system. The available categories include **Standard**, **Fragile**, **Perishable**, **Hazmat**, **Oversized**, and **Liquid**, each corresponding to specific handling, storage, and transportation requirements. Compared with plain enumerations or string-based representations, a strongly typed enumeration provides greater type safety by preventing implicit conversions and invalid values. It also improves code reliability and maintainability by ensuring that all category values are explicitly handled during system operations, thereby reducing the likelihood of runtime errors and simplifying future extensions.

3.2.1.8. Typedef Date

The **Date** alias — defined in terms of the standard library's calendar-date type — provides a concise, semantically meaningful name for the date representation used throughout **LogisticsInfo**. Centralising the alias means that if the underlying date representation ever needs to change, only the alias definition requires updating.

3.2.2. Package Stage Group

This group implements the **State pattern**. Each phase of a package's lifecycle is encapsulated in a dedicated class that implements the **IPackageState** interface. The Package entity itself contains no branching logic about which transitions are valid or what actions should occur on entry; that responsibility belongs entirely to the currently active state object. This separation means that adding a new lifecycle phase requires only a new class — no changes to **Package** or to any other existing state are necessary. Because concrete states are stateless, every one of them also implements a cloning operation (`clone()`), so that Package's copy constructor can duplicate the active state without knowing its concrete type.

3.2.2.1. Interface IPackageState

IPackageState defines the common interface for all concrete package lifecycle states. It specifies the required behaviours that each state implementation must provide,

including state handling, identification, display representation, and cloning. Each concrete state is responsible for determining the conditions for state transitions and updating the package accordingly. The interface also provides a lightweight way to retrieve the current state label and state identifier, supporting efficient display, comparison, and serialization. In addition, the cloning mechanism allows states to be copied independently, ensuring that package duplication does not result in shared state objects and allowing future state implementations to maintain their own internal data safely.

3.2.2.2. Enum PackageStateId

`PackageStateId` is an enumeration that represents the five valid lifecycle phases of a package: ***OnRoute***, ***InStorage***, ***Dispatched***, ***Missing***, and ***Overdue***. Using an enum to represent states avoids runtime type comparisons, simplifies serialization, and enables efficient state handling through value-based comparisons. It also provides a lightweight way for the application layer to identify the current lifecycle phase without interacting directly with the underlying state object. Since these enum values are persisted in storage, their order must remain stable; new states should only be added at the end to prevent compatibility issues with existing stored data.

3.2.2.3. Concrete State Classes

Each concrete state class represents a specific phase in the package lifecycle and encapsulates the rules associated with that phase. Most state transitions are triggered explicitly through warehouse operations, while only the storage state supports automatic transition based on time conditions.

- + **OnRouteState** represents a package that is currently being transported to the warehouse and has not yet been physically received. During this phase, no automatic action is performed, and the transition to storage occurs only after the package is confirmed as received.
- + **InStorageState** represents a package that has been placed in the warehouse. It is the only state with automatic transition behaviour, where the current date is

compared with the expected export date. When the package remains in storage beyond the expected date, it is moved to the overdue state.

- + **DispatchedState** is the terminal state indicating that the package has left the warehouse. No further transitions normally occur from this state.
- + **MissingState** represents a package that cannot be located during a physical inspection. It can be triggered manually from any non-terminal state, and recovery requires human verification before the package is returned to storage.
- + **OverdueState** status denotes a package that has remained in storage beyond its scheduled export date. This status is assigned automatically when the prescribed storage period has elapsed, rather than through direct user action. Once a package is classified as overdue, no further status changes occur automatically. Instead, administrative intervention is required to determine the appropriate course of action. Following the assessment, the package may either proceed to dispatch if it is subsequently exported or be classified as missing if it cannot be located.

3.2.3. Repository and Filter Group

This group separates data persistence and querying responsibilities from the domain model by applying the **Repository** Pattern. The domain logic interacts with storage only through the **IPackageRepository** interface, while the concrete repository implementation is provided externally, allowing the storage mechanism to be changed without affecting the business logic. The system supports multiple persistence approaches, including file-based and database-based implementations, without requiring modifications to the domain classes. Shared conversion utilities ensure that data formats remain consistent across different storage backends.

3.2.3.1. Interface IPackageRepository

The **IPackageRepository** interface defines the abstraction for package persistence by establishing a unified contract for managing package data throughout its lifecycle. It supports the creation, retrieval, modification, and removal of package records while ensuring consistent data management regardless of the underlying storage technology. In addition to basic data management, the interface provides mechanisms for data synchronization, bulk data import and export, and flexible retrieval based on user-defined criteria, enabling efficient data exchange and advanced searching capabilities. By

separating persistence responsibilities from the business logic, the interface improves modularity, maintainability, and extensibility, allowing different storage implementations to be adopted without affecting higher-level components.

3.2.3.2. PackageQueryCriteria

`PackageQueryCriteria` is a value object that encapsulates the optional conditions used to retrieve packages from a repository. Rather than providing separate query operations for each search criterion, it groups multiple filtering attributes into a single composable object, allowing flexible and extensible query construction. The criteria include package information, classification, storage details, weight constraints, date-related conditions, and status-based filters, with each condition applied only when specified. Multiple conditions are combined using logical AND semantics, enabling complex searches through a single query definition. Since the class only represents filtering requirements without performing the actual query, repository implementations remain responsible for interpreting and applying the criteria according to their respective persistence mechanisms. This design improves extensibility and maintainability while preserving a storage-independent querying interface.

3.2.3.3. JsonPackageRepository

`JsonPackageRepository` is a JSON-based implementation of the `IPackageRepository` interface that provides persistent storage for package data using Qt's JSON framework. Package records are loaded into an in-memory collection during initialization, enabling subsequent repository operations and query processing to be performed efficiently without repeated file access. The repository is responsible for synchronizing the in-memory collection with the underlying JSON file while also supporting bulk data import and export in both JSON and CSV formats for data exchange purposes. Serialization and deserialization responsibilities are organized according to the different components of the package model, with shared conversion utilities ensuring consistent representations of common data types such as enumerations and dates. Query requests are evaluated against the in-memory collection by applying all specified filtering

criteria, allowing flexible retrieval while preserving the storage-independent repository interface.

3.2.3.4. `SqlitePackageRepository`

`SqlitePackageRepository` is the database-backed implementation of the **`IPackageRepository`** interface. Unlike the JSON-based repository, it does not maintain an in-memory collection of packages; instead, all operations are executed directly against the SQLite database through a non-owning database connection provided during construction. Therefore, persistence synchronization operations are not required because every modification is applied directly to the database.

Each package is stored as a single row in the database table, with its related value objects such as metadata, addresses, logistics information, and storage location represented as part of the same record. This design is appropriate because these objects do not have independent identities and exist only as components of a package. Data insertion and modification operations use parameterized queries to safely bind values and improve the reliability of database interactions.

The query mechanism differs from the JSON implementation by delegating filtering to the database engine. Each provided search criterion is converted into a corresponding SQL condition within the WHERE clause. Multiple conditions are combined using logical **AND** operations, meaning that a package must satisfy all specified filters to be included in the result. For example, text-based criteria are translated into SQL expressions, while date-based conditions are translated into direct comparisons with the corresponding stored date values. This approach allows the database engine to perform filtering efficiently, avoiding the need to retrieve all records and evaluate them in application memory.

3.2.3.5. `DatabaseConnection`

`DatabaseConnection` is a small RAII wrapper around a single named database connection, constructed once and passed by reference into `SqlitePackageRepository` — the same constructor-injection style already used for the repository and for `WarehouseManager`. On construction it opens the SQLite file (or an in-memory database

for tests), enables foreign-key enforcement, and applies the schema file, treating a duplicate-column error as already-applied rather than fatal so that additive schema changes can be re-run safely on every startup. The destructor closes the connection and removes it from the driver's registry, which matters for running repeated tests in isolation. The class is deliberately not a Singleton: copying and moving are both disabled, and the owning code constructs the one instance it needs, which keeps the class itself trivially testable against an in-memory database.

3.2.4. WarehouseManager

WarehouseManager is the central service class responsible for coordinating business operations related to packages. It manages package workflows through an injected repository abstraction, allowing business logic to remain independent from the underlying storage mechanism. The class takes exclusive ownership of the repository and ensures that a valid repository dependency is provided during initialization. Its responsibilities can be divided into six main areas:

- + **Package Management Operations:** Including basic package management operations, where creating, retrieving, updating, and removing packages are delegated directly to the repository layer.
- + **Package Lifecycle Management:** Handling manual lifecycle transitions. Operations such as receiving, dispatching, marking a package as missing, or restoring a found package first retrieve the target package, validate whether the transition is allowed from its current state, apply the state change, and then persist the updated package. This ensures that lifecycle rules are enforced consistently and prevents invalid state changes.
- + **Automated State Monitoring:** Responsible for periodically evaluating packages whose states require time-based transitions. The manager retrieves only the packages relevant to the monitoring task, allows their current states to evaluate time-dependent conditions, and persists only those packages whose states have changed. This supports automatic transitions such as identifying overdue packages

stored in the warehouse and packages that become late during transportation, while avoiding unnecessary processing.

- + **Package Query and Reporting:** Providing query and reporting functionality. Instead of loading all packages and filtering them within the service layer, the manager constructs appropriate search criteria and delegates filtering to the repository. This keeps the service layer lightweight and allows database-backed repositories to perform filtering efficiently at the storage level. The manager depends only on the repository abstraction and query criteria, keeping package filtering logic independent from the service implementation.
- + **Persistence Management:** The manager exposes persistence operations by delegating data loading and saving to the repository abstraction. This allows the service layer to provide a consistent persistence interface while remaining independent of the underlying storage implementation.
- + **Data Import and Export:** The manager coordinates bulk data transfer by delegating JSON and CSV import and export operations to the repository. After import operations, it ensures that the imported data is synchronized with the underlying storage when required, providing a consistent mechanism for exchanging package data across different persistence implementations.

3.2.5. Presentation Layer (GUI)

The graphical user interface (GUI) layer is implemented using the Qt Widgets framework and constitutes the outermost layer of the system architecture. It is responsible for presenting the application state to users and translating user interactions into corresponding operations within the warehouse management component. This layer depends solely on the underlying application layer to retrieve data and invoke business operations, while the domain, repository, and service layers remain completely independent of any graphical user interface implementation. Such a dependency structure promotes a clear separation of concerns, improves maintainability, and preserves a modular architecture. Furthermore, all GUI-related classes are organized within a dedicated namespace to ensure consistency and facilitate code organization.

3.2.5.1. MainWindow

The **MainWindow** class serves as the primary graphical interface of the Warehouse Management System and acts as the central controller of the presentation layer. Implemented by extending Qt's QMainWindow, it is responsible for coordinating all user interactions while delegating business operations to an injected WarehouseGateway instance. The gateway is supplied through constructor dependency injection rather than being instantiated internally, establishing a clear separation between the graphical user interface and the business logic layer. This design promotes loose coupling, improves maintainability and extensibility, and follows the object-oriented principles of dependency injection and separation of concerns.

The graphical interface adopts a **sidebar-based navigation layout** that organizes the application into four independent functional modules: **Dashboard**, **Package Inventory**, **State Operations**, and **Reports**. These modules are hosted within a QStackedWidget, allowing users to switch seamlessly between different operational views while maintaining a unified application window. Each module encapsulates a distinct warehouse management responsibility, improving modularity and simplifying navigation throughout the system.

The responsibilities of the MainWindow class can be divided into six major functional areas:

- + **User Interface Initialization:** Responsible for constructing the entire graphical interface, including the sidebar navigation, stacked pages, layouts, tables, charts, dialogs, and toolbars. During initialization, the class establishes all required Qt signal–slot connections, creates the periodic monitoring timer, and subscribes to the packagesChanged() signal emitted by the WarehouseGateway. By following the **Observer pattern** through Qt's event mechanism, every modification to warehouse data automatically refreshes all interface components without requiring explicit update calls after each operation.
- + **Package Inventory Management:** Providing complete package management functionality, including browsing, sorting, filtering, creating, editing, removing, importing, and exporting packages. Package creation and modification are

performed through dedicated dialog windows, while all CRUD operations are delegated directly to the WarehouseGateway. Filtering is implemented using PackageQueryCriteria, allowing the gateway to retrieve only matching packages instead of filtering data within the presentation layer. Table sorting is supported through QSortFilterProxyModel, enabling users to organize package information efficiently by clicking table headers.

- + **Dashboard Monitoring:** Displaying an overview of warehouse operations, including package distribution, warehouse occupancy, recent package activities, and daily warehouse tasks. Statistical information is visualized using a pie chart (QPieSeries), while warehouse capacity is represented by a vertical progress bar that changes colour according to predefined occupancy thresholds. The dashboard also provides manual controls for overdue and late package detection and is automatically refreshed whenever package data changes.
- + **Package State Operations:** Handling user-initiated package lifecycle transitions, including receiving inbound packages, dispatching outbound packages, marking packages as missing, and restoring recovered packages. Instead of performing state validation within the graphical interface, the class delegates these operations to the WarehouseGateway, which enforces the corresponding business rules. Operation buttons are dynamically enabled or disabled according to the currently selected package state, preventing invalid state transitions from being initiated through the user interface.
- + **Reporting and Data Visualization:** Presenting warehouse reports, including overdue packages, missing packages, warehouse summaries, and category-based storage statistics. Separate table views display packages requiring attention, while summary panels provide aggregated information that supports warehouse monitoring and operational decision-making. All report data is updated automatically whenever package information changes.
- + **Automated Monitoring and Data Synchronization:** Responsible for maintaining consistency between the graphical interface and warehouse data. A periodic QTimer automatically invokes overdue and late package detection at predefined intervals, ensuring that package lifecycle changes remain synchronized with business rules without requiring user intervention. Import and export

operations support both CSV and JSON formats, enabling data exchange with external systems while relying on the gateway to manage persistence. Through the Observer pattern, every successful business operation triggers automatic interface synchronization, ensuring that all pages always display the latest warehouse information.

Table 3. MainWindow Modules table

Module	Description	Qt Components
Navigation Controller	Manages sidebar navigation and switches between application pages using a stacked layout.	QListWidget, QStackedWidget
Dashboard	Displays package status, warehouse capacity, and recent activities. (1000-package limit with 75% and 90% warning thresholds)	QPieSeries, QProgressBar, QTableView
Package Inventory	Manages package browsing, filtering, and CRUD operations.	QTableView, Filter Panel, Dialogs
State Operations	Handles valid package state transitions based on business rules.	QTableView, Operation Panel
Reports	Displays overdue, missing, and category-based package reports.	QTableView
Automatic Monitoring & Synchronization	Performs periodic overdue/late package checking and refreshes all interface pages when package data changes through the Observer mechanism.	QTimer, Qt Signal–Slot

3.2.5.2. FilterPanel

The **FilterPanel** is a dedicated **QWidget** that provides filtering controls for the **Package Inventory** module. It includes selection controls for package status and category, input fields for storage zone and keyword search, and a button for clearing all active filters. Rather than interacting directly with the warehouse management service, the component is designed solely to collect user-defined filtering criteria and communicate user actions through Qt's **signal–slot mechanism**. This design completely decouples the filtering interface from the business logic, allowing the **Main Window** to receive filter updates and apply the corresponding operations independently. Consequently, the **FilterPanel** functions as a reusable and independent user interface component while the signal–slot communication mechanism provides a practical implementation of the **Observer Pattern**, enabling automatic notification of interface events without introducing direct dependencies between components.

3.2.5.3. PackageTableModel

The **PackageTableModel** serves as the data model connecting the warehouse package collection to Qt's **QTableView**, following the **Model–View** architecture provided by the Qt framework. Implemented by extending **QAbstractTableModel**, it is responsible for adapting domain-level **Package** objects into a tabular representation while keeping presentation concerns separate from the business logic layer. It is reused across multiple interface modules, including the **Package Inventory**, **Dashboard**, **State Operations**, and **Reports** pages, allowing a consistent representation of package information throughout the application. The model defines the table structure by providing row and column counts together with descriptive column headers. It presents nine attributes for each package, namely **ID**, **Name**, **Description**, **Category**, **Weight**, **Zone**, **Status**, **Import Date**, and **Export Date**. Before displaying data, the model converts category enumeration values and custom **Date** objects into user-friendly representations to improve readability. To enable users to identify package conditions quickly, different text colours are applied to the **Status** column, with distinct colours representing **Overdue**, **Missing**, **Dispatched**, and **In Storage** states. Whenever the underlying package collection changes, the **refresh()** function resets the model through

Qt's model notification mechanism, ensuring that all associated table views remain synchronized with the latest warehouse data. Furthermore, it provides direct access to the package represented by a selected table row, allowing other interface components to retrieve the corresponding package information efficiently without performing additional repository queries. In addition to the standard model, the **PackageSmallTableModel** provides a simplified four-column representation consisting of **Name**, **Category**, **Zone**, and **Status**, making it suitable for compact interface components such as dashboards and report panels while maintaining the same state-based visual presentation.

3.2.5.4. AddPackageDialog

The **AddPackageDialog** is implemented as a modal **QDialog** responsible for collecting information required to create a new package. The interface organizes user input into four tabs, namely **Metadata**, **Location**, **Address**, and **Logistics**, improving usability by grouping related attributes. Users provide package metadata, storage location, source and destination information, and logistics dates through Qt input widgets such as **QLineEdit**, **QComboBox**, **QSpinBox**, **QDoubleSpinBox**, and **QDateEdit**. After confirmation, the dialog converts the collected values into corresponding domain objects and constructs a new **Package** instance through the domain factory method. The dialog focuses solely on gathering user input, while package validation and business processing are delegated to the warehouse management service. The dialog provides standard **OK** and **Cancel** actions through a **QDialogButtonBox**, while several fields are initialized with default values to simplify package creation.

3.2.5.5. EditPackageDialog

The **EditPackageDialog** is implemented as a modal **QDialog** responsible for modifying an existing package. Unlike the add dialog, all editable fields are automatically pre-populated using the selected package, allowing users to update package metadata, storage location, and logistics information without re-entering unchanged values. The interface is organized into three tabs: **Metadata**, **Location**, and **Logistics**, while immutable information such as package identity and addresses remains unchanged. After confirmation, the dialog creates an updated copy of the selected package by replacing the

modified value objects before returning it to the presentation layer for further processing. Similar to the add dialog, business validation and persistence are handled outside the dialog, maintaining a clear separation between the user interface and business logic. The dialog preserves immutable package information, including the package identifier, addresses, and current state, while updating only mutable value objects before returning the modified package.

3.2.5.6. WarehouseGateway

The **WarehouseGateway** acts as an intermediary between the graphical user interface and the **WarehouseManager**, following the **Observer Pattern** by serving as the system's Subject. Read-only operations simply forward data requests to the service layer without notifying observers, whereas operations that modify package data, including package creation, update, removal, receiving, dispatching, marking packages as missing, and restoring recovered packages, notify registered interface components only after the corresponding business operation has been completed successfully. If an operation fails due to a business rule violation, no notification is issued, ensuring that the user interface remains consistent with the actual system state. Automatic overdue and late package detection follow the same principle by notifying observers only when at least one package changes state. To simplify communication and reduce coupling, the gateway employs a single notification event for all successful data modifications, allowing individual interface components to refresh only the information they require. In addition to package management operations, the gateway provides persistence support through save and load operations, as well as bulk data exchange using CSV and JSON import/export. Export operations are treated as read-only and therefore do not trigger notifications, whereas load and import operations emit notifications after successfully updating the warehouse data. By exposing a unified interface to the presentation layer, the gateway encapsulates the underlying **WarehouseManager** and prevents graphical components from interacting directly with the business logic. Meanwhile, the **AddPackageDialog** and **EditPackageDialog**, implemented as Qt **QDialog** components, are responsible for collecting or updating package metadata, addresses, logistics information, and storage location before forwarding the resulting package data to the service layer through the **WarehouseGateway** for processing.

3.2.5.7. Data flow summary

During application startup, the system initializes the warehouse data and synchronizes package states to ensure that overdue and other time-dependent conditions are updated before the user begins interacting with the application. The graphical interface then retrieves the latest warehouse information and presents it through the Dashboard, Package Inventory, State Operations, and Reports modules, providing a consistent view of the current warehouse status.

During normal operation, user actions such as creating, editing, removing, receiving, dispatching, reporting missing packages, or restoring recovered packages are processed by the business layer. After each successful operation, the updated package information is propagated to the presentation layer, allowing all interface modules to display the latest warehouse data automatically. Read-only operations, such as browsing, filtering, searching, and generating reports, retrieve the required information without modifying the underlying data. Any business rule violations or processing errors are reported to the user through the application's error-handling mechanism, ensuring data consistency and reliable warehouse operations.

3.2.6. Relationship Summary

The Package Data group contains the core domain entities, where **Package** acts as the aggregate root and composes its associated value objects, including package metadata, addresses, storage location, logistics information, and the current package state. The Package State group implements the **State Pattern**, allowing each lifecycle stage to encapsulate its own behavior through the **IPackageState** interface. The Repository and Filter group abstracts data persistence and querying using the **Repository Pattern**, supporting both JSON and SQLite implementations through a common interface. The Service Layer centralizes business logic within the **WarehouseManager**, which performs package validation, lifecycle management, and query processing while communicating with the repository through **Dependency Injection**. Finally, the GUI communicates with the service layer through the **WarehouseGateway**, while Qt's **Model–View architecture** and **signal–slot mechanism** support efficient data presentation and event-driven

synchronization, maintaining a clear separation between the presentation layer and business logic.

3.3. Application flow or main use-case flow.

This section traces the principal use-case flows through the system, illustrating how the classes described above collaborate at runtime.

3.3.1. New Package Addition

The process of adding a new package begins when the user opens the **Add Package** dialog from the graphical interface and enters the required package information, including metadata, storage location, addresses, and logistics details. After the user confirms the input, the dialog constructs a new **Package** object and forwards it through the **Main Window** to the **WarehouseGateway**. The gateway delegates the request to the **WarehouseManager**, which validates the package according to the defined business rules before persisting it through the repository layer. Once the package has been successfully stored, the graphical interface is automatically synchronized so that the newly added package becomes immediately visible in the Dashboard, Package Inventory, State Operations, and Reports modules. This workflow clearly separates user interaction, business processing, and data persistence while maintaining consistency across the application.

3.3.2. Lifecycle State Transitions

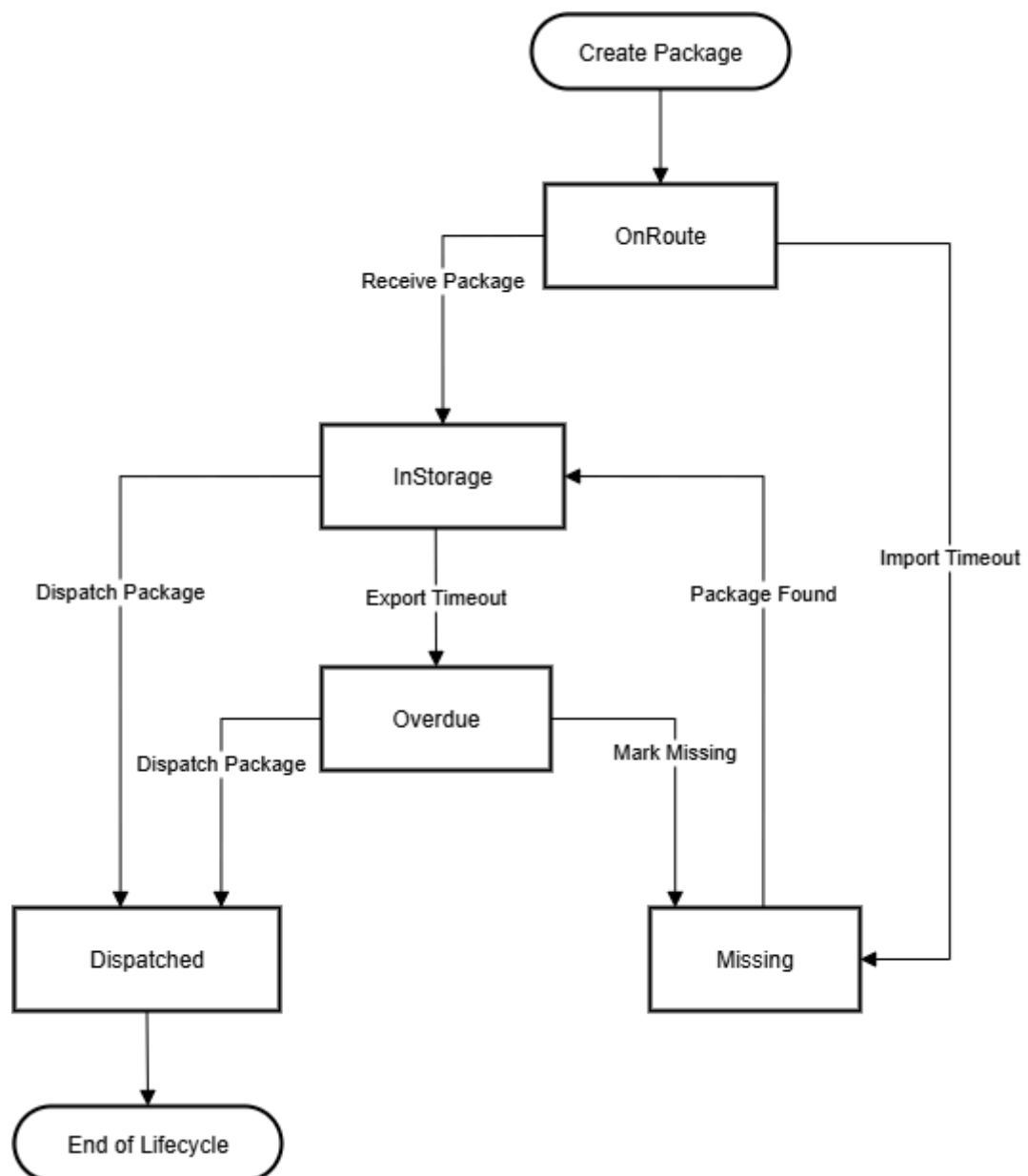


Image 2. State Lifecycle

Automatic state transitions are intentionally limited within the package lifecycle. The only automatic transition occurs when a package in the **In Storage** state exceeds its expected export date, causing it to enter the **Overdue** state. This transition is triggered automatically during overdue checking without requiring user intervention.

All other state transitions are initiated through warehouse operations. Before a transition is applied, the system verifies that the requested operation is valid for the

package's current state. Invalid operations are rejected, while valid transitions update the package state and persist the modified information.

The supported lifecycle transitions are as follows. A package moves from **On Route** to **In Storage** after its arrival at the warehouse is confirmed. Packages in **In Storage** or **Overdue** can be dispatched, resulting in the **Dispatched** state. Packages that cannot be located during warehouse operations may be marked as **Missing** from any non-terminal state, while recovered packages are restored from **Missing** to **In Storage**.

State-specific behavior is encapsulated within the corresponding state classes, whereas the coordination and validation of lifecycle operations are performed by the **WarehouseManager**. This separation of responsibilities follows the **State Pattern**, allowing individual state behaviors and business workflow logic to evolve independently.

3.3.3. Querying and Filtering

The query mechanism is built around **PackageQueryCriteria**, which encapsulates all supported filtering conditions within a single query object. Instead of providing multiple query methods with different parameter combinations, the system accepts one criteria object whose optional attributes specify the desired constraints. Unspecified attributes are ignored during query execution, allowing the same mechanism to support both simple and complex searches. When multiple criteria are provided, they are combined using a logical AND relationship so that only packages satisfying all specified conditions are returned.

The **WarehouseManager** delegates query execution to the repository layer, enabling the underlying persistence implementation to determine the most appropriate retrieval strategy. This design allows different repository implementations to provide equivalent query functionality while remaining independent of the service layer. To simplify common operations, the service layer also provides convenience methods for frequently used queries, such as searching by package identifier, category, storage location, or package state. For scenarios where package data has already been retrieved into memory, **PackageFilter** offers an alternative mechanism for applying additional filtering without accessing the repository again.

By encapsulating query conditions within a dedicated value object and delegating query execution to the repository, the design improves extensibility, reduces interface complexity, and maintains a clear separation between business logic and data access.

3.3.4. Packages Update and Removal

Updating or removing a package follows a consistent workflow coordinated by the **WarehouseManager**. For update operations, the target package is first retrieved, the required information or state changes are applied, and the modified package is then submitted for persistence. For removal operations, the package identified by its unique identifier is deleted from the repository. In both cases, the **WarehouseManager** delegates the actual data persistence to the **IPackageRepository** interface without depending on any specific storage implementation. This design ensures that business logic remains independent of the underlying persistence mechanism while providing a consistent and extensible approach to maintaining warehouse data.

3.3.5. GUI synchronization

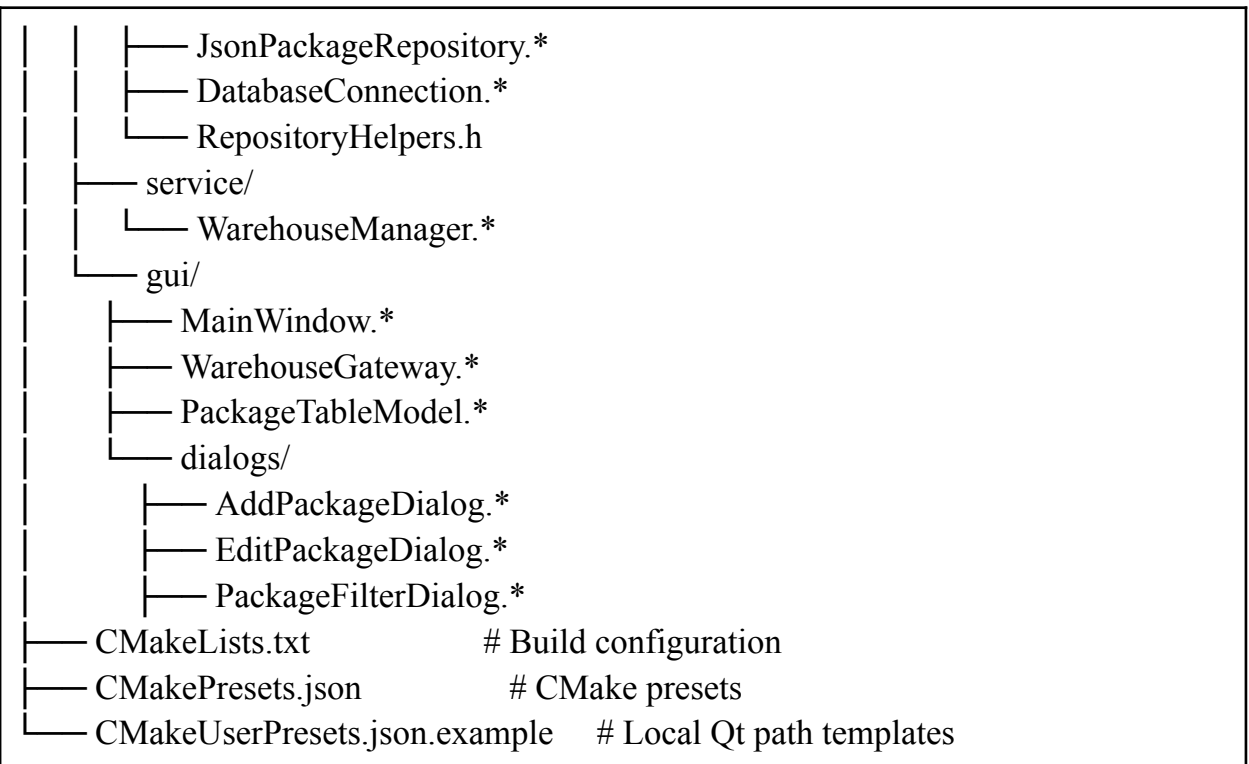
Whenever a business operation modifies warehouse data, such as adding, updating, removing, receiving, dispatching, or changing the state of a package, the graphical interface is automatically synchronized with the latest system state. The **WarehouseGateway** notifies the presentation layer after successful data modifications, allowing the Dashboard, Package Inventory, State Operations, and Reports modules to retrieve updated information from the service layer and refresh their displayed data. Read-only operations, such as browsing, filtering, and querying packages, do not trigger synchronization because they do not modify the underlying warehouse data. This event-driven communication mechanism follows the **Observer Pattern**, ensuring loose coupling between the presentation layer and business logic while maintaining consistent information across all interface modules.

4. PROJECT ARCHITECTURE

4.1. Project Structure

Table 4. Folder Structure

—	main.cpp	# Application entry point
—	docs/	
—	QTINSTRUCTION.md	# Qt setup / environment instructions
—	GUIDELINE.md	# Project / coding guidelines
—	resources/	
—	theme.qss	# Application stylesheet
—	icons.qrc	# Qt resource collection for icons
—	icons/	# SVG icons (sort, chevrons, etc.)
—	data/packages.json	# Sample / seed package data
—	db/schema.sql	# SQLite database schema
—	src/	
—	domain/	
—	entities/	
—	Address.h	
—	Category.h	
—	Date.h	
—	Dimension.h	
—	LogisticsInfo.h	
—	Package.h	
—	Package.cpp	
—	PackageMetadata.h	
—	StorageLocation.h	
—	states/	
—	IPackageState.h	
—	PackageStateId.h	
—	InStorageState.*	
—	OnRouteState.*	
—	DispatchedState.*	
—	MissingState.*	
—	OverdueState.*	
—	queries/	
—	PackageQueryCriteria.h	
—	repository/	
—	IPackageRepository.h	
—	SqlitePackageRepository.*	



The build system is based on CMake (minimum version 3.23), configured through the root-level CMakeLists.txt. The project is compiled using the C++20 standard and automatically enables Qt's AUTOMOC, AUTOUI, and AUTORCC features for processing QObject classes, user interface files, and Qt resources. Required Qt6 modules (Core, Widgets, Gui, Charts, and Sql) are located through a three-step fallback mechanism that first checks an externally defined CMAKE_PREFIX_PATH, then the QT_BIN_DIR environment variable, and finally auto-detects the default Windows Qt installation directory. Source files are collected automatically using file(GLOB_RECURSE) for each project module, while application resources such as icons are embedded using qt_add_resources. On Windows, windeployqt is executed after compilation to deploy the required Qt runtime libraries. If the database schema file (resources/db/schema.sql) exists, it is copied automatically beside the executable through a POST_BUILD command. An optional BUILD_TESTS configuration flag is also provided to enable future test integration.

4.2. Architecture

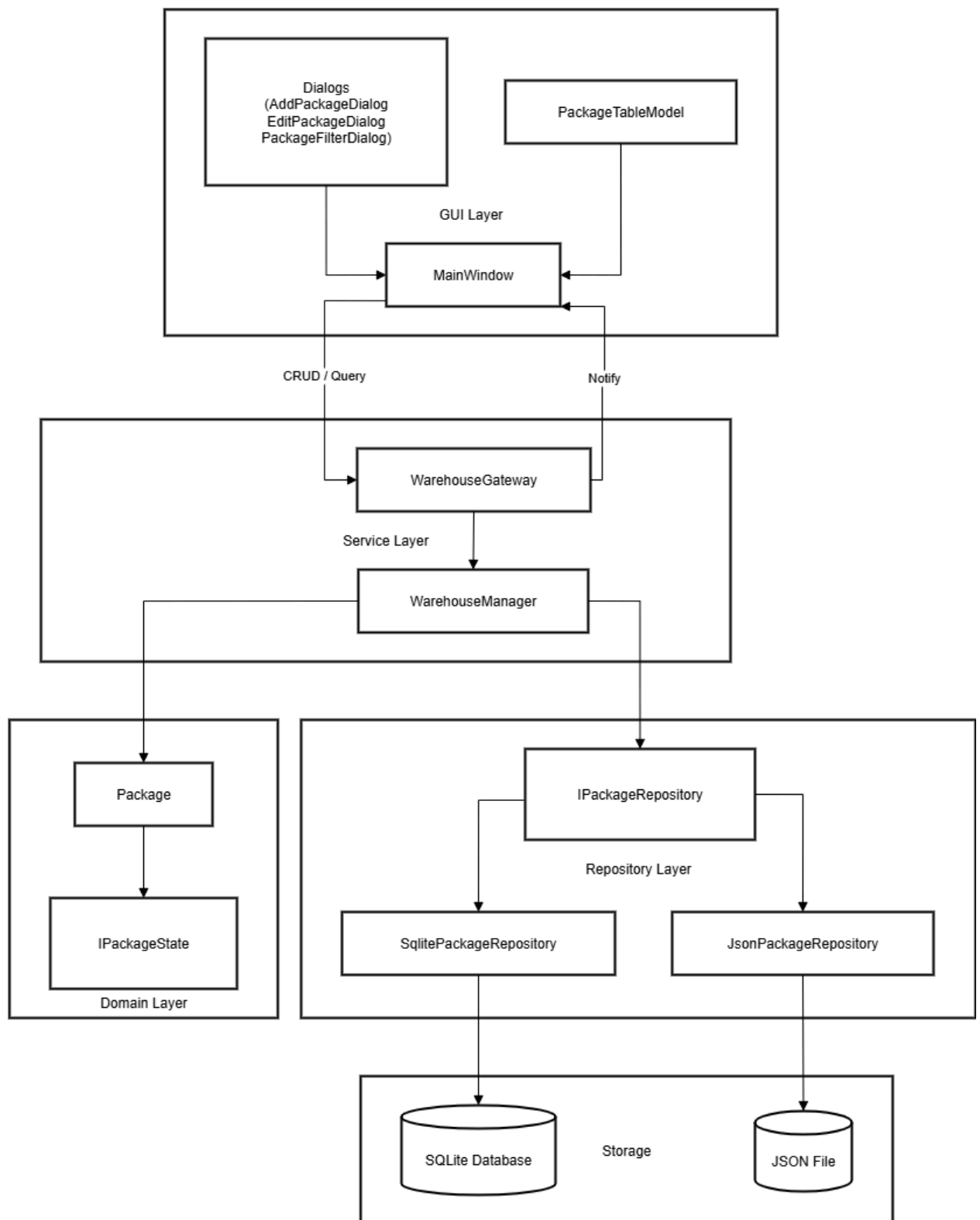


Image 3. Warehouse Management Architecture

The system adopts a layered architecture with a strict and unidirectional dependency flow from the **presentation layer** to the **service layer**, followed by the **repository layer**, and finally the **domain layer**. The presentation layer consists of the

graphical user interface components, including the **MainWindow**, **WarehouseGateway**, **PackageTableModel**, **FilterPanel**, and package management dialogs. User requests are forwarded through the **WarehouseGateway**, which communicates with the **WarehouseManager** and notifies registered interface components whenever the system state changes, providing an implementation of the **Observer Pattern**. The **domain layer** contains the core business entities, value objects, and the package lifecycle state model, remaining independent of the Qt framework. The **repository layer** abstracts data persistence through the **IPackageRepository** interface, allowing both **JsonPackageRepository** and **SqlitePackageRepository** to be used interchangeably. The **service layer**, represented by the **WarehouseManager** and **PackageFilter**, encapsulates the application's business rules and remains independent of the underlying persistence implementation through dependency injection.

4.3. Design Patterns

Table 5. Project's Design Patterns

Pattern	Location	Suitability and Benefits
State	IPackageState and its concrete state classes	Package behaviour changes according to its lifecycle state. It eliminates complex conditional statements, encapsulates state-specific behaviour, and simplifies the addition of new states.
Observer	WarehouseGateway and Qt's signal-slot mechanism	Multiple GUI components must remain synchronized after data changes. It decouples the user interface from the business logic and automatically updates all

		registered views while reducing direct dependencies.
Repository	IPackageRepository abstraction; JsonPackageRepository and SqlitePackageRepository implementations	The system supports multiple persistence mechanisms. It separates data access from business logic, allowing storage implementations to be replaced without affecting higher layers.
Dependency Injection	WarehouseManager receives IPackageRepository via its constructor; MainWindow receives WarehouseManager* via its constructor	Dependencies are supplied externally rather than created internally. It reduces coupling, improves flexibility, and facilitates unit testing with alternative implementations.
Query Object	PackageQueryCriteria, translated independently by SqlitePackageRepository::buildWhereClause() and JsonPackageRepository::findByCriteria()	Search conditions vary by caller and must work identically across two storage backends. Representing the query as plain optional fields, translated per-backend, avoids one-method-per-filter API bloat (findByState, findByZone, ...) while letting each repository optimise its own translation (SQL WHERE clause vs. in-memory scan).

5. IMPLEMENTATION AND TESTING

5.1. Main modules

Table 6. Main modules

Module	Directory	Main Components
Domain	src/domain	Package, value objects, IPackageState, and concrete state classes
Repository	src/repository	IPackageRepository, JsonPackageRepository, SqlitePackageRepository, DatabaseConnection, and RepositoryHelpers
Service	src/service	WarehouseManager
GUI	src/gui	MainWindow, WarehouseGateway, PackageTableModel, FilterPanel, and Dialogs

5.2. Data storage method

The system supports two interchangeable persistence backends behind IPackageRepository:

- + **JSON Repository:** stores package information in a human-readable JSON file while maintaining an in-memory collection to provide efficient data access during application execution. This implementation is lightweight, easy to inspect, and well suited for testing, debugging, and small-scale deployments.
- + **SQLite Repository:** stores package information in a relational database, where package attributes are organized into a structured table with indexes on frequently

queried fields to improve retrieval performance. It serves as the default persistence mechanism for the graphical application, providing efficient data management, scalability, and reliable storage while supporting future database extensions.

5.3. Important features

- There are five core features:
- + **Package Management:** The system supports complete **CRUD (Create, Read, Update, Delete)** operations through the **Package Inventory** module. A confirmation dialog is displayed before package deletion to prevent accidental data loss.
- + **Package State Management:** Package state transitions are validated according to predefined business rules, ensuring that only valid lifecycle transitions can be performed while preventing invalid operations.
- + **Automatic Overdue Detection:** The system automatically identifies overdue packages by periodically checking packages currently in storage. This process is executed during application startup and repeated hourly to ensure package states remain up to date.
- + **Dashboard Monitoring:** The **Dashboard** provides a real-time overview of warehouse operations by visualizing package distribution across different states and displaying warehouse capacity with warning thresholds at **75%** and **90%** of the maximum capacity of **1000 packages**.
- + **Search and Filtering:** The system supports multi-criteria filtering based on package status, category, storage zone, and keywords, enabling efficient package retrieval and inventory management.

5.4. Test cases and results

Table 7. Test cases and results

ID	Feature	Description	Input	Expected Result	Actual Result	Pass/ Fail

1	Add Package	Add a valid package	Complete form, export date > import date	New package appears in Inventory, state = On Route	New package appears in Inventory, state = On Route	Pass
2	Add Package	Export date earlier than import date	importDate = Aug 10, exportDate = Aug 5	Error shown, package not created	Error shown, package not created	Pass
3	Edit Package	Update metadata/location	Change name and zone of an existing package	Data updates correctly, state unchanged	Data updates correctly, state unchanged	Pass
4	Edit Package	Update export date earlier than import date	importDate = Aug 10, exportDate = Aug 5	Error shown, package not edited	Error shown, package not edited	Pass
5	Remove Package	Delete an existing package	Select 1 package, confirm delete	Package removed from the list	Package removed from the list	Pass
6	State Transition	Receive (On Route → In Storage)	Package currently On Route	State changes to In Storage	State changes to In Storage	Pass
7	State Transition	Receive when package is not On Route	Package currently In Storage	Receive when package is	Receive when package is not On Route	Pass

				not On Route		
8	State Transition	Dispatch (In Storage/Overdue → Dispatched)	Package currently In Storage	State changes to Dispatched	State changes to Dispatched	Pass
9	Auto-Transition	Package past its export deadline	expectedExportDate ≤ today, state In Storage	Automatically transitions to Overdue after "Check Overdue"	Automatically transitions to Overdue after "Check Overdue"	Pass
10	Auto-Transition	Package late arriving	importDate ≤ today, state On Route	Automatically transitions to Missing after "Check Late"	Automatically transitions to Missing after "Check Late"	Pass
11	Filter Packages	Filter by multiple combined criteria (state + category + zone)	state=InStorage, category=Freight, agile, zone=A	Only packages matching all 3 conditions shown	Only packages matching all 3 conditions shown	Pass
12	Export/Import CSV	Export then re-import	CSV file exported	Imported data matches	Imported data matches original 100%	Pass

			from the system	original 100%		
13	Export/Import JSON	Export then re-import	JSON file exported from the system	Imported data matches original 100%	Imported data matches original 100%	Pass
14	Import CSV/JSON with Partial Data	Import with limited data	Import a CSV or JSON file where only specific packages are present	Only specific packages are modified or added	Only specific packages are modified or added	Pass
15	Backend Consistency	Compare filter results on SQLite vs JSON	Same criteria, two different backends	Results are identical	Results are identical	Pass
16	Dashboard	Pie chart & capacity statistics	N packages in various states	Chart figures match the actual data	Chart figures match the actual data	Pass

5.5. Bugs found and fixes

Table 8. Bugs and fixes

Bugs	Fix
Using std::string in	Changed std::string to QString in

JsonPackageRepository.cpp caused string handling issues.	JsonPackageRepository.cpp.
Package received a new ID after reload.	Updated Package create() and load() to call construction logic and keep the same ID after reload.
Build error caused by undeclared fromQt() function.	Replaced fromQt() with proper inline conversion code.
CMake used absolute paths and dateFromString returned the wrong type.	Added package query support for future filters.
CMakeSetting did not support plugin path and applySchema() lacked comment stripping.	Expanded CMakeSetting to hold plugin path and added comment stripping for applySchema().
Package copy action did not use state clone().	Updated all package copy actions to use state clone().
CMakeSetting was used instead of CMakePresets, and members could not create their own presets based on examples.	Replaced CMakeSetting with CMakePresets and added support for members to create custom presets based on examples.

6. AI USAGE AND INDIVIDUAL ACCOUNTABILITY

6.1. AI tools

Table 9. AI tools usage

AI tools	Usage
Claude	Review the source code for potential hidden issues, identify areas for improvement, and provide recommendations for enhancing the system's structure, performance, and maintainability.
Gemini	Obtain suggestions for report formatting, including document structure, content organization, and presentation style to ensure a clear and professional academic layout.
ChatGPT	Refine technical descriptions by transforming code-oriented language into natural, coherent, and academically appropriate writing. It was also used for proofreading and improving the clarity and readability of the report.
Github Copilot	Use GitHub Copilot to generate implementation instructions and coding guidelines, providing suggestions for development tasks and code structure. All generated recommendations were reviewed, modified where necessary, and verified by the student before use
Antigravity	Use Antigravity (Google DeepMind) to identify obsolete filter-related source files after the refactoring and to diagnose a build issue caused by stale CMake cache entries. The student verified the AI's suggestions before deleting the files, reconfigured the build system, and confirmed that the project compiled successfully.

6.2. Individual AI usage log of each member

6.3. Individual contribution

 [01 - Contribution]

7. CHALLENGES AND FUTURE ENHANCEMENTS

7.1. Encountered Problems.

During the development process, several technical challenges were encountered. One of the primary issues involved maintaining a portable Qt6 configuration across heterogeneous development environments, as team members had different Qt6 installation paths on their machines. To address this, the CMakeLists.txt file was designed with a fallback discovery mechanism that first honours an externally set CMAKE_PREFIX_PATH (e.g. from CMakeSettings.json, CMakeUserPresets.json, or a command-line flag), then falls back to the QT_BIN_DIR environment variable if set, then searches the default Windows Qt installer location by auto-detecting the newest installed version, and finally emits a descriptive error message if none of these locate a valid Qt installation. This approach eliminated the need for machine-specific configuration changes while improving the portability of the build system.

Another challenge concerned the deployment of the Qt SQL plugin. On some development machines, the SQLITE database driver could not be loaded because the required Qt SQL plugin was not deployed together with the application. This was mitigated by ensuring the build's post-build step runs windeployqt, which bundles the SQL plugin alongside the executable; the CMake configuration still emits a warning rather than failing the build if windeployqt cannot be located, so the issue can still recur on machines without windeployqt available on PATH.

Maintaining the project's four-layer architecture throughout collaborative development also required careful coordination. In particular, preserving the independence of the domain layer from the Qt framework while multiple team members modified the codebase necessitated continuous refactoring. To improve architectural consistency and reduce code duplication, common utility functions for enum and date conversion shared by JsonPackageRepository and SqlitePackageRepository were

extracted into `RepositoryHelpers.h`, resulting in improved code reuse, maintainability, and adherence to the intended software architecture.

7.2. Limitations of the current version.

- **Lack of Automated Testing:** The current implementation does not include an automated unit testing framework. Although a testing module is reserved within the project structure, it has not yet been implemented.
- **Incomplete Database Constraints:** The SQLite database schema does not currently enforce a foreign key constraint for the `container_id` field because the **Container** module has not yet been developed.
- **No User Authentication and Authorization:** The system does not provide user authentication or role-based access control. Consequently, **Inventory Managers** and **Warehouse Staff** share the same interface and system privileges.
- **Limited JSON Repository Performance:** The JSON repository performs filtering by scanning the entire in-memory package collection, which may lead to reduced performance as the dataset grows. In contrast, the SQLite repository benefits from indexed database queries, providing more efficient data retrieval for large-scale datasets.

7.3. Possible future improvements.

Several enhancements are planned for future development. The system can be extended by implementing the **Container** and **Employee** modules, together with foreign key constraints for `container_id` to improve data integrity. In addition, a comprehensive unit testing framework should be introduced to verify the correctness of the domain, service, and repository layers. The application can also be enhanced by incorporating user authentication and role-based access control to distinguish the responsibilities of **Inventory Managers** and **Warehouse Staff**. Finally, the performance of the **JSON** repository can be improved through additional indexing mechanisms, while the **SQLite**

repository should remain the recommended persistence backend for large-scale warehouse deployments.

REFERENCES

- [1] T. Gaddis, *Starting Out with C++: From Control Structures through Objects*, 9th ed. Boston, MA, USA: Pearson, 2021.
- [2] E. Freeman and T. Sierra, *Head First Design Patterns*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [3] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley Professional, 2002.
- [4] J. Horton, *Sams Teach Yourself C++ in One Hour a Day*, 8th ed. Indianapolis, IN, USA: Sams Publishing, 2019.